

Demo Guide — Live Presentation Script

This document is **for the presenter only**. Use it to run the live demo and to answer questions confidently. It is not part of the deliverable to colleagues.

Before you walk into the room

1. Open the `PRE_DEMO_CHECKLIST.md` and tick every box.
 2. Have these three browser tabs ready (do not open them during the demo — that wastes time):
 3. `http://localhost:8000` (the running webapp)
 4. `https://github.com/venkatesbonthala2932/GLM-OCR` (your repo)
 5. The PDF report open in Preview (`tests/benchmark/OCR_REPORT.pdf`)
 6. Have the terminal ready with `python webapp.py` already running and `[startup] model ready` already printed. **Do not start the model load in front of the audience — it takes 15–30 s and looks bad.**
 7. Pick **two sample images** beforehand to demo with — one from the repo (`examples/source/code.png` is reliable) and one of your own (your choice). Have both on the Desktop ready to drag in.
-

Demo flow (10 minutes total)

1. Open with the problem (1 minute)

"Our requirement is local OCR — no cloud APIs, no per-page costs, no sensitive documents leaving the network. I evaluated several open- source options and chose GLM-OCR because it's the current #1 on the OmniDocBench benchmark and is fully Apache-licensed."

2. Show what's running (1 minute)

Switch to the browser tab at `http://localhost:8000`.

"This is the local web interface. It's running on my workstation, talking to the GLM-OCR model loaded into the GPU. No internet connection is required at this point — I can demonstrate that."

(If asked) Show the terminal with `[startup] model ready` printed.

3. Demo upload — image (2 minutes)

Drag `examples/source/code.png` into the drop zone.

"I'll drop a code screenshot first. The model resizes the image to a safe maximum, then runs inference on the Mac's MPS GPU."

Click **Run OCR**. While it runs:

"Average time is about 5–60 seconds depending on document complexity. Code screenshots are usually fast because the layout is regular."

When the result appears, point out: - Indentation preserved. - Symbols (`{`, `;`, `<`) read correctly.

4. Demo upload — PDF (2 minutes)

Reset the page. Drag any PDF (use `examples/source/GLM-4.5V.pdf` if you don't have one of your own).

"PDFs are handled by splitting each page into an image and running OCR per page. You get one result block per page so you can see exactly which page succeeded and which failed."

Click **Run OCR**. Note that the larger PDF will take longer; if it's slow, talk through the architecture while it runs:

"Each page goes through PyMuPDF to render at 200 DPI, then PIL for the image conversion, then transformers for the model call. The entire path is in `webapp.py` — about 330 lines, single file."

5. Show the benchmark (2 minutes)

Switch to the PDF report (`OCR_REPORT.pdf`) or the rendered Markdown.

"To validate the deployment I built a repeatable benchmark across eight document categories — code, scientific paper, table, handwriting, document seal, multi-page PDF. All eight passed, 0 failures, average 45 seconds per page."

Scroll through the results table. Optionally show one or two of the detailed outputs to demonstrate Chinese-character handling.

6. Show the limitations (1 minute) — *do not skip this*

"Honest assessment: the model is trained primarily on Chinese and English. Indian languages like Hindi, Tamil, Telugu are not reliably supported. For those workloads I'd pair this with PaddleOCR or Tesseract as a fallback engine. That's the clear next step."

Showing limitations builds trust. Managers respect honesty more than inflated claims.

7. Wrap with deliverables (1 minute)

Switch to the GitHub tab.

"Everything is version-controlled on this repository. `SETUP.md` walks a new user through installation in about 20 minutes. There's a formal project report in `docs/`, a benchmark report under `tests/`, and the full source. Happy to take questions."

Anticipated questions and answers

These are the questions a technical manager or senior engineer is most likely to ask. Read these out loud at least once before the demo so you can repeat them in your own words.

Q1. "Why this model and not Tesseract / Google Vision / AWS?"

"Tesseract is free but weak on real-world photos and modern layouts. Cloud APIs are accurate but introduce per-page cost, network dependency, and data-residency concerns. GLM-OCR is the strongest open-source option benchmarked today, runs entirely offline after the initial model download, and is Apache 2.0 licensed."

Q2. "How accurate is it really?"

"On clean printed text — both Chinese and English — accuracy is near 100 % in the benchmark cases. On handwriting it depends on neatness; printed-style handwriting works well, cursive less so. On Indian scripts it currently does not work reliably."

Q3. "How long does each page take?"

"Average 45 seconds on the test hardware (Apple M-series, MPS GPU). Small images like stamps are 2–3 seconds; large multi-column pages with formulas can be 90–100 seconds. The bottleneck is the model autoregressive generation."

Q4. "What's the cost?"

"Zero recurring cost. One-time storage: 2.5 GB for the model. CPU / GPU is whatever workstation it runs on. We could deploy it on a shared internal GPU server later for higher throughput."

Q5. "Can it handle 100 pages per hour? 1000? 10000?"

"On this single workstation, roughly 80 pages per hour. For higher throughput we'd need either GPU parallelism (multiple workers on a bigger card) or a smaller distilled model. The bottleneck is purely inference time — the rest of the pipeline is fast."

Q6. "Why is it not authenticated?"

"Today it binds to localhost on my workstation, which is the right default for development. Before exposing it on the network we'd add HTTP basic auth or place it behind an internal SSO proxy — that's noted in the limitations section of the report."

Q7. "How do I run this on my own machine?"

"SETUP.md walks through it. Clone the repo, create a venv, run two `pip install` commands, run `python webapp.py`. Total time about 20 minutes including the model download."

Q8. "What if the model gets a wrong answer?"

"There's no automatic confidence score from this model. For production use I'd recommend either a human-in-the-loop UI for reviewing low-stakes outputs, or running two engines (this + a different one) and flagging disagreement. Both are doable but not built yet."

Q9. "Is this code yours, or did you use AI?"

"I built it using Claude Code as a coding assistant — that's a common workflow now. I made the architecture decisions, tested every change end-to-end on this hardware, identified the limitations, and own the repository. I can walk through any file in detail and explain why each design choice was made."

(This is the honest answer. Don't hide it. Most managers in 2026 see AI-assisted work as a positive signal of modern engineering practice.)

Q10. "What would you build next?"

"Three things, in priority order. One, add PaddleOCR as a fallback engine for Indian languages. Two, persist OCR results to SQLite so they're searchable. Three, package the entire stack as a Docker image so teammates don't need to set up Python."

Q11. "What happens if HuggingFace goes down?"

"Only the first download needs HuggingFace. After the 2.5 GB cache is built, the system runs fully offline. We could also mirror the model on an internal artefact store for full air-gap deployment."

Q12. "How is the model accuracy measured?"

"The benchmark report tracks output length and the full OCR text per sample. We don't have a labelled ground-truth set today, so accuracy is currently inspected manually. Building a labelled internal test set would be a worthwhile next step before any production rollout."

What to do if something fails live

Failure	What to say	What to do
Webapp won't start	"The model takes 15 seconds to load — let me show the report while it boots."	Switch to the PDF report tab, talk while it loads
Upload returns blank	"This one hit our known Indian-language limitation — let me show one in English instead."	Drop a sample from <code>examples/source/</code>
Browser tab refresh issue	(silent) refresh and continue	Cmd+R, keep going
Out-of-memory error	"We're hitting the GPU memory cap on this large file. The system has a setting to scale image size — I'll skip this and use a smaller sample."	Drop a smaller image
You don't know an answer	"Good question — I haven't tested that specifically. Let me get back to you with a clear answer."	Write it down, follow up by email

Never bluff. Saying "I don't know, I'll find out" is far better than guessing wrong in front of senior people. They will respect the honesty.

Closing line options

Pick one in advance:

- "Happy to share the repository link with anyone who wants to try it."
 - "If anyone has a specific document type they'd like me to test, I can run it through after this meeting."
 - "I'd like to scope the multilingual extension as the next iteration if the team agrees this direction is useful."
-

After the demo

1. Send everyone the repository link.
2. Send the PDF (`PROJECT_REPORT.pdf`) as a follow-up attachment.
3. Write down every question you got that you couldn't answer immediately, and respond by email within 24 hours.

That follow-through is what builds long-term trust with senior colleagues. Far more than the demo itself.